

# ④ physics informed neural Net... (Raissi)...

Page  $\Rightarrow$  2, 3, 13 (Relevant stuff)

$$\Rightarrow u_t + \mathcal{N}[u; \lambda] = 0, x \in \Omega, t \in [0, T]$$

$u(t, x) \rightarrow$  latent (hidden) sol'n

$\mathcal{N}[\cdot; \lambda] \rightarrow$  non linear operators parameterized by  $\lambda$   
 $\lambda$  subset of  $\mathbb{R}^D$

$\hookrightarrow$  this setup encapsulate range of machine physics  
like cons. laws, diffusion processes, advection-  
diffusion-reaction systems & kinetic eqs

Operator  $\rightarrow$  is a function that help produce another function:

lin  $\rightarrow$  satisfies linearity  $\Rightarrow L(au+bv) = aL(u) + bL(v)$

Non-linear doesn't satisfy lin. eg.  $N(u) = u \frac{\partial u}{\partial n}$

(ex)  $\Rightarrow$  D boundary corner. to the:

$$N[u; \lambda] = \lambda_1 u_{xx} - \lambda_2 u_{nn} \text{ \& } \lambda = (\lambda_1, \lambda_2)$$

① Data-driven soln of PDE

$$u_t + N[u] = 0, x \in \Omega, t \in [0, T]$$

①  $\rightarrow$  conti. time models  
 $\rightarrow$  discrete time models

① Conti. time models

( $\rightarrow$   $f(t, n)$  on left)

$$f := u_t + N[u]$$

app  $\Rightarrow u(t, n)$  by N.N

(ex) Boundary

①  $f(x, t)$  or  $f(t, n) \Rightarrow$  no mathematical diff  
 $\hookrightarrow$  just staying consistent with it

$$u_t + u_{nn} - (0.01/\pi) u_{nn} = 0, n \in [-1, 1],$$

$$u(0, n) = -\sin(\pi n)$$

$$t \in [0, 1]$$

$$u(t, -1) = u(t, 1) = 0$$

$$f := u_t + u_{nn} - (0.01/\pi) u_{nn}$$

$$u(t, n) \Rightarrow u(t, n; w, b) \left\{ \begin{array}{l} \text{by auto diff} \\ f(t, n; w, b) \end{array} \right.$$

② Resulting pair of  $u(t, n; w, b)$  &  $f(t, n; w, b)$   
 must satisfy boundary regardless of choice of  $w$  &  $b$ .

③ Summary

NN predicts  $\Rightarrow u(t, n) \rightarrow$  auto diff  $\Rightarrow u_t, u_n, u_{nn}$

$$\hookrightarrow u_t + u_{nn} - v u_{nn} = 0$$

$$\text{defined } \Rightarrow f(t, n) = u_t + u_{nn} - v u_{nn}$$

& train network so that

$u_t$  given  $\Rightarrow$  known initial / boundary values &  
 it PDE residual  $f$  is close to zero

$$\Rightarrow \{t_i, n_i, u_i\}_{i=1}^{N_u} \Rightarrow$$

Let  $t=0, x=0.3, u = -\sin(\pi(0.3))$ .

↳ one training data pt.  $= (t_i, x_i, u_i)$

or  $(t_u^2, x_u^2, u^2)$

So  $\Rightarrow \{t_i, x_i, u_i\}_{i=1}^{N_u}$  initial & boundary training data

means "We have  $N_u$  pts. where we know the value of  $u$ ".

### ① Initial & boundary training data

↳ means we know sol<sup>n</sup> only on the places where the problem specification gives us information

↳ Boundary  $\Rightarrow u(0, x) = -\sin(\pi x)$

↳ i.c

So we can take pts such as:

$(0, -0.8), (0, -0.4), (0, 0), (0, 0.4), (0, 0.8)$

↳ we must connect  $u$  values at each other

↳ i.c training pts.

Here  $u(t, -1) = 0$   
&  $u(t, 1) = 0$

So  $\Rightarrow (0.2, -1, 0), (0.5, -1, 0), (0.8, -1, 0)$

↳ Boundary training pts.

### ② Initial Boundary data

↳  $f_0 = u(0, x) = -\sin(\pi x)$   
&  $u(t \pm 1) = 0$

### ③ PDE Collocation pts:

↳  $(t_i^*, x_i^*) \dots (t_N^*, x_N^*)$

↳ then  $\rightarrow$  Does my NN satisfy PDE here?

Calc  $\Rightarrow f = u_t + u_{xx} - \nu u_{xx}$   
& try to make  $f \approx 0$

Target  $= 0$

$f_0(t, x)$

$MSE_L = \frac{1}{N_f} \sum_{i=1}^{N_f} |f_0(t_i^*, x_i^*)|^2$

Date: / /

PDE  $\rightarrow$  Elliptic  
 $\rightarrow$  Parabolic  
 $\rightarrow$  Hyperbolic

① often describes something like a steady spatial eqn.

$\hookrightarrow u_{xx} = \text{known function}$

② associated with diffusion/smoothing & time evolution

ex  $u_t = \nu u_{xx}$

③ associated with transport/propagation of info & wave-like or convection-dominated behaviour

Burgers  $\rightarrow$  non-linear  $\Rightarrow$   $u u_x$   
 transport term

Q1  $\nu \rightarrow 0$  eqn app  $\Rightarrow u_t + u u_x = 0$

Date: / /

② Suppose we make up a function ourselves:

ex  $u(t, x) = e^{-t} \sin(x)$

diff  $u_t, u_x, u_{xx}$

fun calc: What  $f(t, x)$  would need to be

so we could construct PDE for which our invented  $u(t, x)$  is guaranteed to be exact soln

$\hookrightarrow$  Idea is called manufactured soln

ex  $f(t, x) = u_t + u u_x - \nu u_{xx}$

fun  $\Rightarrow u_t + u u_x - \nu u_{xx} - f = 0$

$\hookrightarrow$  so we have the ans. From the begg.

③ But we can't do this for our case we will use the established Burgers problem.

②  $u(t, n)$ .

↳ instead of explicitly with the links let a NN. repre. it.

$$\Rightarrow u(t, n) \rightarrow u(t, n; w, b)$$

$$\hookrightarrow u_0(t, n), \theta = \{w, b\}.$$

$w, b = 1$  initially they are random.

$\therefore$  NN produces some random funt<sup>n</sup>.

↳ training modifies those para

$\Rightarrow u_0(t, n)$   $\rightarrow$  the output NN gives

We need  $\Rightarrow$   $u_t, u_n, u_{nn}$

in/2nd of manually deriving the entire NN eqn; auto diff calculates these derivatives with imp.

Python

$$\hookrightarrow y = \text{network}(t, n)$$

$$u_t \neq \text{deri. of } u \text{ wrt } t$$

$$u_n = u \text{ wrt } n$$

$$u_{nn} = u \cdot n \text{ wrt } n$$

$$f = u_t + u \cdot u_n - n u \cdot u_{nn}$$

① before trying the net. on a given set of data:...

↳ at the pbm way & Bi. a. no optimum yet.

$$f_0 = u_0 \cdot t + u_0 \cdot u_n - n u_0 \cdot u_{nn}$$

↳ so whatever the current weights are,  $f_0$  is always the res. comp. to Net.  $u_0$ .

$f_0 = L[u_0]$   $\rightarrow$  is built into how we define  $f$ .

$f_0 \rightarrow$

$w, b = \text{random values}$

$w^*, b^* = \text{found values}$

$u(t, n; w^*, b^*)$

↳ this is what N.W. belongs

W. b. chs  
to

model known data  
 $u(0, x) \approx -\sin(\pi x)$   
 $u(t, \pm 1) \approx 0$

Satisfy then  $\rightarrow f(t, x) \approx 0$   
 PDE

"approx. will conserve mass & momentum  
 within the accuracy of resi. loss"

we do not do

④ Solve PDE res  $\Rightarrow$  both satisfy of gov. physics

④ Conti. & discrete problem

origins  
 $\hookrightarrow$  PDE is written for every pt. in domain

$$\textcircled{a} \quad u_t + u u_x - \nu u_{xx} = 0$$

for:  $-1 \leq x \leq 1$ ,  $0 \leq t \leq 1$

④ many possible pts.

① Discrete pts

$\hookrightarrow$  comp. can't fit every calc. as no. of pts.

$\therefore$  classical num. methods create a finite repr.

② divide space into:

$x_0 \dots x_1 \dots x_2 \dots x_3 \dots x_n$

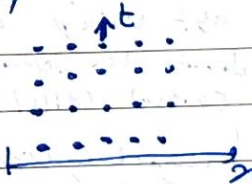
$\hookrightarrow$  now we only work with those discrete locations/elements/cells.

$\hookrightarrow$  that's discretization.

③ "spatio-temporal discretization"

$x \rightarrow$  spatial,  $t \rightarrow$  temporal

① dividing  $x$  &  $t \rightarrow$  finite pieces



$\Rightarrow$  each dot (grid pt.)  
 represents a finite  
 computational location

Date: / /

Buyer - motivation continues

Date: / /

they (Kassir) didn't use this:

instead  $\Rightarrow U_0(t, n) \rightarrow$  conti. NN function

(i)  $(t, n) = (0.37, 0.481) \rightarrow$  it produces a value  
 $\hookrightarrow$  different  $n$  & different value

(ii) no req. that there locates belong to a fixed mesh.

(But)  $\rightarrow$  they still use finite set of colloc. pts during training.

" They don't discretize the PDE into a conventional mesh-based num. solvers.

FEM

FVM

1---e---|---e2---|---e3---

| cell 1 | cell 2 | cell 3 | cell 4 |

$\hookrightarrow$  within each element, soln is approx. using basis functions.

$\hookrightarrow$  instead of primarily using abt. values at pts, FVM divide the domain into control cells/volumes

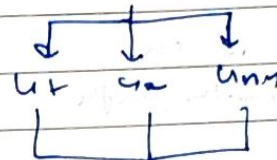
$\hookrightarrow$  write a conservation eqn for each cell

change in vol = Flux in - Flux out

approx. PDE using finite elements

NN  
 $U_0(t, n; w, b)$

Auto diff



$$f = u + v + w - v + w$$

PDE loss  $\rightarrow f^2$

known  $T_{in}$  / Boundary data

IC + BC loss

Total loss  
 PDE + IC + BC

Optimize  $w, b$

toward  $U_0(t, n)$

After training

Learned  $U_0$  errors

① We don't tell NN because exact ref. soln

Rule 2  $\rightarrow$  obey I.C

$U_0(0, n) \approx \sin(\pi n)$

Rule 3  $\rightarrow$  obey B.C

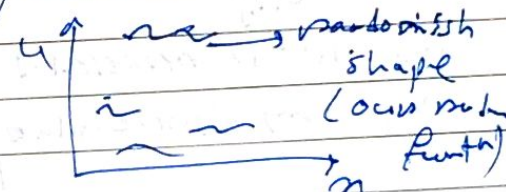
$U_0(t, -1) \approx 0$

$U_0(t, 1) \approx 0$

Rule 3  $\rightarrow$  obey Boundary

$U_0(t, -1) + U_0(t, 1) - 2U_0(t, 0) \approx 0$

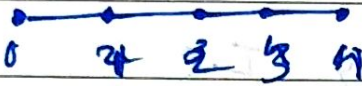
event  $\rightarrow U_0(t, n) \approx U_{true}(t, n)$



Date: / /

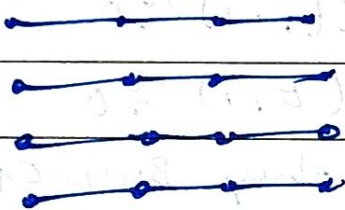
## FDM

$n-1$



on multiple time levels

at



if we can calc. approx. values at those discrete locations

unknown might then be

$u_{i,j}$

Continuous is represented

by many discrete values

## P2M

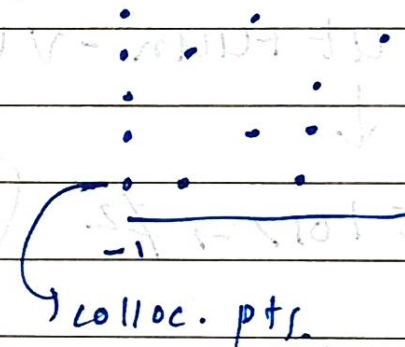
interp.

$$u(t, x) = N(u)$$

if network is only contr. nodes further

at

1.0



colloc. pts.

$$calc = f_0(t_i, u_i)$$

Calculation

for min. period

But underlying network still can be evaluated b/w dots

Cont. Numerical  
Function

+ Finite colloc.  
Sampler